

Tscan

Plateforme d'analyse, de corrélation et de validation
de vulnérabilités de sécurité

Cahier des charges

Version 0.1 — Document vivant, mis à jour au fil du projet

Projet réalisé dans le cadre d'un stage à l'**ANTIC**

(Agence Nationale des Technologies de l'Information et de la Communication)

Document rédigé progressivement — Phase d'étude, de définition du produit
et de conception, préalable au développement

12 août 2026

Contents

1	Introduction et contexte	2
1.1	Contexte général	2
1.2	Problématique	2
1.3	Justification du projet	2
1.4	Cadre d'utilisation et responsabilité	3
2	Objectifs	4
2.1	Objectif général	4
2.2	Objectifs spécifiques	4
3	Utilisateurs et acteurs	6
3.1	Utilisateurs cibles	6
3.1.1	Analyste en cybersécurité (utilisateur principal)	6
3.1.2	Responsable informatique / décideur (utilisateur secondaire)	6
3.2	Acteurs secondaires	6
3.3	Synthèse des cas d'usage principaux	7
4	Étude de l'existant et analyse comparative	8
4.1	Méthodologie de l'étude	8
4.2	Panorama des scanners existants	8
4.2.1	Constat central	8
4.3	Plateformes d'agrégation et de gestion de vulnérabilités	9
4.4	Approches de validation active et de réduction des faux positifs	9
4.5	Sources de connaissances en vulnérabilités	9
4.6	Formats d'échange de résultats	10
4.7	Structure des rapports professionnels	10
5	Limites des solutions existantes et positionnement de Tscan	11
5.1	Synthèse des limites identifiées	11
5.2	Positionnement de Tscan	11
5.3	Réponse à la question centrale	11
6	Besoins fonctionnels	13
6.1	Module Import et analyse de résultats externes	13
6.2	Module Corrélation, validation et scoring	14
6.3	Module Scan actif (web)	15
6.4	Module Reporting	15
6.5	Module Application et connaissances	16
7	Besoins non fonctionnels	17
7.1	Performance	17
7.2	Sécurité	17
7.3	Portabilité et compatibilité	17

7.4	Maintenabilité et évolutivité	17
7.5	Utilisabilité	18
7.6	Disponibilité hors-ligne	18
8	Contraintes	19
8.1	Contraintes de temps et de ressources	19
8.2	Contraintes techniques	19
8.3	Contraintes organisationnelles	19
8.4	Contraintes légales et éthiques	19
9	Exigences de sécurité	21
9.1	Autorisation et périmètre	21
9.2	Traçabilité et journalisation	21
9.3	Protection des données collectées	21
9.4	Sécurité de l'outil lui-même	22
10	Architecture prévisionnelle	23
10.1	Vue d'ensemble	23
10.2	Décision d'architecture — Partage de la logique entre CLI et application desktop	23
10.3	Décision d'architecture — Découplage moteur / règles / connaissances	24
10.4	Modèle de données pivot	24
10.5	Gestion des preuves	25
11	Technologies envisagées	26
11.1	Langage du cœur	26
11.2	Application desktop	26
11.3	Synthèse de la pile technologique retenue	26
11.4	Autres points de vigilance	27
12	Méthodologie de développement	28
12.1	Principe général	28
12.2	Développement incrémental	28
12.3	Gestion des blocages	28
12.4	Documentation continue	28
12.5	Gestion de version	28
12.6	Revue régulières	29
13	Planning	30
13.1	Cadrage général	30
13.2	Répartition par semaine	31
13.3	Jalon de sécurité	31
13.4	Suivi d'avancement	32
14	Critères de réussite du MVP	33
15	Limites explicites du MVP	34
16	Perspectives post-stage	35

Chapter 1

Introduction et contexte

1.1 Contexte général

Ce document constitue mon cahier des charges pour le projet **Tscan**, que je réalise dans le cadre d'un stage effectué au sein de l'ANTIC. Contrairement à un projet académique classique, je conçois Tscan dès l'origine avec l'ambition qu'il puisse être potentiellement utilisé de manière opérationnelle par l'ANTIC, et qu'il constitue la base d'une évolution ultérieure vers un produit à vocation commerciale.

Le projet démarre sans historique : aucun code, aucune architecture, aucune base de données, aucun dépôt de code et aucun cahier des charges n'existaient avant la rédaction de ce document. L'ensemble du projet est donc construit méthodiquement, en partant d'une étude de l'existant, avant toute décision technologique ou architecturale.

1.2 Problématique

La détection de vulnérabilités de sécurité repose aujourd'hui sur une combinaison d'outils spécialisés, chacun couvrant une couche particulière du système d'information (réseau, application web, dépendances logicielles, conteneurs). Cette fragmentation impose à l'analyste de sécurité :

- d'utiliser plusieurs outils dont les résultats sont produits dans des formats hétérogènes et non directement comparables ;
- de trier manuellement un volume important de résultats, parmi lesquels une proportion significative constitue des faux positifs ;
- de recouper lui-même les informations issues de différents outils pour confirmer ou infirmer une vulnérabilité ;
- de produire, souvent manuellement, un rapport structuré compréhensible à la fois par des experts techniques et par des décideurs non techniques.

Ce travail de tri et de confirmation est chronophage, répétitif, sujet à l'erreur humaine, et peu capitalisé d'un scan à l'autre : les décisions de classification (faux positif, vulnérabilité confirmée) ne sont, dans la plupart des flux de travail artisanaux, pas conservées ni réutilisées systématiquement.

1.3 Justification du projet

L'étude de l'existant menée en amont de ce document (détaillée au chapitre 4) montre qu'il n'existe pas, dans l'écosystème actuel des outils accessibles, de solution qui combine à la fois :

- (a) l'import et la normalisation des résultats de plusieurs scanners hétérogènes ;
- (b) une tentative de confirmation active des vulnérabilités, contrôlée et non destructive, plutôt qu'une simple déduplication ;
- (c) un système de corrélation et de scoring de confiance explicable, avec conservation systématique des preuves ;
- (d) une production de rapport professionnel exploitable par un analyste et par un responsable informatique.

C'est cet espace, situé entre les scanners spécialisés et les plateformes de gestion de vulnérabilités existantes, que Tscan a vocation à occuper.

1.4 Cadre d'utilisation et responsabilité

Tscan est un outil de sécurité offensive à usage strictement autorisé. Toutes les fonctionnalités de scan actif sont conçues pour ne s'exécuter que sur des cibles pour lesquelles l'utilisateur dispose d'une autorisation explicite, avec des mécanismes de limitation de périmètre (cible, profondeur, durée, types de tests) et une priorité donnée aux tests non destructifs. Ce principe structure l'ensemble des choix fonctionnels et architecturaux décrits dans ce document.

Chapter 2

Objectifs

2.1 Objectif général

Concevoir et développer un logiciel professionnel de cybersécurité, **Tscan**, capable d'améliorer la qualité de la détection de vulnérabilités en réduisant le taux de faux positifs, à travers l'import et l'analyse de résultats de scanners externes ainsi que l'exécution d'un scan actif ciblé, tout en produisant des rapports exploitables par des profils techniques et non techniques.

2.2 Objectifs spécifiques

- O1. Import multi-formats** — permettre l'import de résultats provenant de plusieurs scanners externes (a minima Nuclei et OWASP ZAP pour le MVP), et normaliser ces résultats dans un modèle de données unique, tout en conservant le résultat original pour garantir la traçabilité.
- O2. Corrélation et réduction des faux positifs** — croiser les informations disponibles (résultats multiples, contexte de la cible, base de connaissances) afin d'attribuer à chaque résultat un statut de validation (Confirmée, Probable, Potentiel faux positif, Faux positif) et un niveau de confiance explicable.
- O3. Validation active non destructive** — pour un périmètre défini de familles de vulnérabilités, tenter une confirmation technique contrôlée, sûre et non destructive, plutôt que de se limiter à une inférence statique.
- O4. Scan actif ciblé** — exécuter, sur des cibles web autorisées, une phase de reconnaissance puis une détection portant sur un nombre volontairement restreint mais fiable de familles de vulnérabilités.
- O5. Système de règles extensible** — structurer la logique de détection et de validation sous forme de règles versionnées et documentées, indépendantes du cœur du moteur, afin de permettre leur enrichissement progressif sans modification du code central.
- O6. Base de connaissances évolutive** — intégrer et maintenir à jour des sources de vulnérabilités reconnues (CVE/NVD, CWE, CISA KEV) et permettre un fonctionnement partiellement hors-ligne.
- O7. Reporting professionnel** — produire des rapports structurés (résumé exécutif et détails techniques), incluant preuves, niveaux de confiance et recommandations sourcées.

O8. Architecture évolutive — concevoir une architecture modulaire permettant, après le stage, l’ajout de nouveaux formats, de nouvelles règles, de nouvelles sources de connaissances et de nouvelles interfaces, sans remise en cause du cœur du système.

Non-objectif explicite
Tscan ne vise pas, y compris à terme, à détecter l’intégralité des familles de vulnérabilités existantes ni à concurrencer directement des solutions commerciales matures. L’objectif est un socle extensible, initialement restreint mais fiable, conçu pour être enrichi progressivement.

Chapter 3

Utilisateurs et acteurs

3.1 Utilisateurs cibles

3.1.1 Analyste en cybersécurité (utilisateur principal)

Profil. Personnel technique de l'ANTIC ou d'une équipe de sécurité, en charge de l'évaluation de vulnérabilités sur des systèmes autorisés.

Besoins. Importer des résultats de scan bruts sans perdre l'information d'origine ; obtenir un tri fiable entre vulnérabilités réelles et faux positifs ; disposer de preuves techniques exploitables ; pouvoir corriger manuellement une classification automatique ; lancer un scan ciblé rapide sur une cible précise ; disposer d'une interface suffisamment technique (CLI incluse) pour s'intégrer à son flux de travail.

3.1.2 Responsable informatique / décideur (utilisateur secondaire)

Profil. Personne en charge de la remédiation ou de la décision, pas nécessairement experte en cybersécurité.

Besoins. Comprendre rapidement le niveau de risque global d'une cible ; disposer d'un résumé exécutif clair, sans jargon excessif ; connaître les priorités de correction ; suivre l'évolution du niveau de risque entre deux évaluations.

3.2 Acteurs secondaires

- **Encadreur de stage / ANTIC** : validation des orientations, bénéficiaire potentiel de l'outil, évaluateur du projet.
- **Moi-même (développeur)** : conception, développement, tests et documentation, en autonomie complète sur la durée du stage.
- **Cibles scannées** : systèmes et applications faisant l'objet d'une autorisation explicite d'évaluation — acteur passif mais central dans la définition des contraintes de sécurité et de non-nuisance.

3.3 Synthèse des cas d'usage principaux

Cas d'usage	Description
UC1 — Import et analyse	L'analyste importe un résultat de scanner externe (Nuclei, ZAP...) ; Tscan normalise, corrèle, valide et classe les résultats.
UC2 — Scan direct	L'analyste lance un scan Tscan sur une cible web autorisée ; Tscan effectue reconnaissance, détection, validation et scoring.
UC3 — Correction manuelle	L'analyste consulte un résultat, ses preuves et son niveau de confiance, et corrige manuellement son statut si nécessaire.
UC4 — Génération de rapport	L'analyste génère un rapport structuré à partir des résultats validés, destiné à être partagé avec un responsable informatique.
UC5 — Mise à jour des connaissances	Le système met à jour, lorsqu'une connexion est disponible, sa base de connaissances (CVE, CWE, CISA KEV) et ses règles de détection.

Chapter 4

Étude de l'existant et analyse comparative

4.1 Méthodologie de l'étude

Cette étude vise à répondre à la question centrale : *pourquoi Tscan doit-il exister alors que des solutions reconnues existent déjà ?* Elle s'appuie sur l'analyse de solutions open source et commerciales représentatives de chaque grande approche : scanners réseau, scanners web (DAST), moteurs de détection basés sur des templates, plateformes d'agrégation et de gestion de vulnérabilités, ainsi que sur les formats et sources de données associés.

4.2 Panorama des scanners existants

Le paysage actuel des scanners de vulnérabilités ne comporte pas d'outil unique couvrant l'ensemble des besoins : chaque outil couvre une couche spécifique, et la pratique courante consiste à combiner plusieurs outils complémentaires plutôt qu'à s'appuyer sur une seule solution.

Outil	Catégorie	Points forts	Limites
Nmap	Découverte réseau	Cartographie fiable des ports et services, base de nombreux autres outils	N'est pas en soi un scanner de vulnérabilités
Nuclei	Détection basée sur templates	Rapide, templates précis, faible taux de faux positifs pour les CVE cataloguées	Limité aux vulnérabilités déjà décrites par un template ; ne découvre rien de nouveau
OWASP ZAP	DAST web (actif et passif)	Couverture large des vulnérabilités applicatives, crawling intégré, open source actif	Scans longs (jusqu'à plusieurs heures), résultats de scan actif nécessitant une vérification manuelle avant confiance
OpenVAS / Greenbone	Scanner réseau traditionnel	Couverture large, feed de vulnérabilités important, solution mature	Outil lourd, plus lent, orienté infrastructure plutôt qu'applicatif web
Nessus	Scanner réseau (commercial)	Référence historique du secteur, large couverture	Solution propriétaire, licence payante
Trivy	Scanner de dépendances / conteneurs	Rapide, ciblé sur les composants logiciels connus	Hors périmètre du scan actif web

4.2.1 Constat central

Le contraste entre Nuclei (précis mais étroit) et ZAP (large mais bruyant) illustre un compromis structurel de l'écosystème actuel : plus un outil élargit sa couverture de détection, plus le taux de

résultats nécessitant une vérification humaine augmente. Aucun des deux outils, ni leur simple juxtaposition, ne résout ce compromis de façon automatisée.

4.3 Plateformes d'agrégation et de gestion de vulnérabilités

Des plateformes telles que **DefectDojo** apportent une réponse partielle à la fragmentation des résultats : elles centralisent les résultats de plusieurs scanners dans un modèle de données commun, appliquent une déduplication basée sur des empreintes calculées à partir de champs spécifiques à chaque scanner, et mémorisent les décisions de triage des analystes pour éviter de re-signaler un résultat déjà classé comme faux positif.

Ce mécanisme est pertinent mais reste, par construction, **passif** : la déduplication et la mémorisation des décisions humaines ne constituent pas une tentative de confirmation technique de la vulnérabilité elle-même. Le dédoublement entre outils différents y est d'ailleurs volontairement limité par défaut, la comparabilité entre les résultats de scanners différents étant jugée peu fiable sans configuration fine.

4.4 Approches de validation active et de réduction des faux positifs

Certaines solutions commerciales (par exemple les technologies dites de *proof-based scanning*) vont plus loin en tentant une exploitation contrôlée, sûre et non destructive de la vulnérabilité détectée, afin de ne remonter un résultat comme confirmé que lorsqu'une preuve concrète a été obtenue (réponse serveur, comportement observable). D'autres outils DAST s'appuient sur le rejeu de requêtes et l'analyse contextuelle pour statuer sur l'exploitabilité réelle d'un résultat avant de le signaler.

Ces approches démontrent la faisabilité technique d'une confirmation active non destructive, mais restent implémentées dans des solutions propriétaires et fermées, non interopérables avec les résultats d'autres scanners.

Sur le plan méthodologique, il est également établi que des approches fondées sur l'apprentissage automatique pour la réduction des faux positifs exigent un volume important de données étiquetées, restent peu explicables, et présentent un risque de faux négatifs difficilement acceptable dans un contexte de sécurité. Ce constat conforte le choix, retenu pour Tscan, de privilégier une approche déterministe (règles, corrélation, scoring explicable) plutôt qu'un recours prématuré à l'intelligence artificielle.

4.5 Sources de connaissances en vulnérabilités

L'étude comparative des principales sources publiques de renseignement sur les vulnérabilités fait ressortir une complémentarité plutôt qu'une hiérarchie stricte entre elles :

Source	Rôle principal	Accès
CVE / NVD (NIST)	Registre de référence des vulnérabilités connues, scores CVSS, classification CWE et CPE associée	API publique, gratuite
CISA KEV	Liste des vulnérabilités confirmées comme activement exploitées — signal fort de priorisation	API publique, gratuite
CWE (MITRE)	Taxonomie des types de faiblesses, indépendante des instances CVE — utile pour catégoriser les règles de détection	Public
GitHub Security Advisories	Avis de sécurité curés, complémentaires à NVD sur l'écosystème logiciel libre	API GitHub, interface web

Un principe méthodologique important, retenu pour la conception de la base de connaissances de Tscan, est que la déduplication doit se faire par **vulnérabilité réelle** et non par identifiant brut, une même faiblesse pouvant être référencée sous plusieurs identifiants provenant de sources différentes.

4.6 Formats d'échange de résultats

Il existe un format standardisé et largement adopté pour l'échange de résultats d'analyse : **SARIF** (Static Analysis Results Interchange Format), normalisé par l'OASIS, qui structure en JSON les résultats d'outils d'analyse afin de permettre leur agrégation et leur comparaison. Ce format est cependant historiquement orienté vers l'analyse statique de code, et n'est pas nativement adapté aux résultats de scanners réseau ou web dynamiques. Il constitue néanmoins une cible pertinente pour l'export de résultats produits par Tscan, en complément d'un import fondé sur les formats natifs des scanners ciblés (Nuclei, ZAP).

4.7 Structure des rapports professionnels

L'analyse de la pratique du secteur (rapports de test d'intrusion, d'évaluation de vulnérabilités) fait ressortir une structure récurrente, indépendante de l'outil utilisé : un résumé exécutif court et sans jargon destiné aux décideurs, une description de la portée et de la méthodologie, le détail technique de chaque résultat avec preuves et références (CVSS, CWE), des recommandations de remédiation prioritaires, et — lorsque pertinent — une comparaison avec les évaluations précédentes. Cette structure sert de référence directe pour la conception du module de reporting de Tscan.

Chapter 5

Limites des solutions existantes et positionnement de Tscan

5.1 Synthèse des limites identifiées

L'étude de l'existant met en évidence trois limites structurelles dans l'écosystème actuel :

- L1. Fragmentation des formats et des outils** — aucun outil ne couvre l'ensemble du besoin, et la combinaison d'outils multiples ne s'accompagne pas nativement d'une normalisation et d'une corrélation des résultats.
- L2. Réduction des faux positifs majoritairement passive** — les plateformes d'agrégation existantes déduplique et mémorisent les décisions humaines, mais ne tentent pas, par elles-mêmes, de confirmer techniquement une vulnérabilité importée.
- L3. Confirmation active réservée aux solutions propriétaires fermées** — les rares outils capables de validation active non destructive sont commerciaux, non interopérables avec des résultats externes, et ne produisent pas de scoring de confiance explicable croisant plusieurs sources.

5.2 Positionnement de Tscan

Proposition de valeur de Tscan
Tscan se positionne à l'intersection de ces trois limites : un système capable d'importer et de normaliser les résultats de plusieurs scanners externes, de tenter une confirmation active non destructive sur un périmètre de vulnérabilités maîtrisé, et de produire un score de confiance explicable et traçable, restitué dans un rapport exploitable aussi bien par un analyste que par un décideur.

Ce positionnement ne vise pas à remplacer les scanners existants, mais à constituer une couche complémentaire de fiabilisation et de synthèse, exploitable dès le MVP sur un périmètre volontairement restreint (scan actif web), et conçue pour être étendue progressivement après le stage.

5.3 Réponse à la question centrale

Pourquoi Tscan doit-il exister alors que ces solutions existent déjà ? Parce qu'aucune des solutions étudiées ne combine, dans un même outil ouvert et interopérable : l'import multi-sources, la tentative de confirmation active non destructive, et un scoring de confiance explicable et tracé

— alors que chacun de ces éléments est individuellement démontré comme faisable par au moins une solution existante étudiée dans ce chapitre.

Chapter 6

Besoins fonctionnels

Les besoins fonctionnels sont regroupés par module et identifiés par un code (RF-xx) afin de faciliter leur suivi tout au long du projet et leur traçabilité vers les tests. Chaque besoin est associé à un statut MVP (retenu pour la version minimale démontrable) ou Post-MVP (prévu dans l'architecture mais développé après le stage), tel que défini au chapitre 2 et détaillé lors de la phase de définition du produit.

6.1 Module Import et analyse de résultats externes

Code	Besoin	Statut
RF-01	Importer un fichier de résultats au format Nuclei (JSONL).	MVP
RF-02	Importer un fichier de résultats au format OWASP ZAP (JSON).	MVP
RF-03	Importer un fichier de résultats au format Nessus (.nessus / XML).	MVP si le planning le permet
RF-04	Normaliser tout résultat importé dans un modèle de données pivot unique (« Finding »).	MVP
RF-05	Conserver le résultat brut d'origine associé à chaque résultat normalisé, à des fins de traçabilité.	MVP
RF-06	Exporter les résultats au format SARIF.	Post-MVP

6.2 Module Corrélation, validation et scoring

Code	Besoin	Statut
RF-07	Attribuer à chaque résultat un statut parmi : Confirmée, Probable, Potentiel faux positif, Faux positif.	MVP
RF-08	Appliquer un pré-filtrage contextuel (service actif, version, cohérence) avant toute autre analyse.	MVP
RF-09	Corréler plusieurs résultats portant sur la même cible provenant de sources différentes.	MVP
RF-10	Calculer un score de confiance explicable pour chaque résultat, avec le détail des critères ayant contribué au score.	MVP
RF-11	Permettre à l'utilisateur de consulter les preuves associées à un résultat (requêtes, réponses, captures).	MVP
RF-12	Permettre à l'utilisateur de corriger manuellement le statut d'un résultat, en conservant la trace de la modification (statut automatique initial, statut corrigé, auteur, date).	MVP
RF-13	Structurer les règles de détection et de validation sous forme d'entités versionnées, indépendantes du cœur du moteur.	MVP (fondation)
RF-14	Faire évoluer automatiquement les règles à partir des corrections effectuées par les analystes.	Post-MVP

6.3 Module Scan actif (web)

Code	Besoin	Statut
RF-15	Effectuer une phase de reconnaissance sur une cible web autorisée (serveur, en-têtes, TLS, technologies détectables).	MVP
RF-16	Identifier les technologies/CMS/frameworks courants exposés par la cible.	MVP (périmètre réduit)
RF-17	Détecter les vulnérabilités relevant de la Security Misconfiguration (dont Clickjacking).	MVP
RF-18	Détecter les composants logiciels vulnérables connus, par correspondance entre version détectée et base de connaissances CVE.	MVP
RF-19	Détecter les vulnérabilités de type Cross-Site Scripting réfléchi.	MVP
RF-20	Détecter les cas basiques de Broken Access Control (panneaux d'administration exposés, listing de répertoires, endpoints sensibles non protégés).	MVP
RF-21	Détecter l'absence de protection CSRF sur les formulaires sensibles.	MVP
RF-22	Détecter les injections SQL basées sur les erreurs (error-based).	Bonus MVP (optionnel selon avancement)
RF-23	Tenter une confirmation active non destructive pour chacune des familles ci-dessus, avant attribution du statut final.	MVP
RF-24	Permettre à l'utilisateur de définir un périmètre de scan (cible, profondeur, durée, types de tests autorisés).	MVP
RF-25	Effectuer un scan authentifié (avec session utilisateur).	Post-MVP
RF-26	Effectuer un scan ciblé sur une spécification d'API (OpenAPI...).	Post-MVP

6.4 Module Reporting

Code	Besoin	Statut
RF-27	Générer un rapport structuré comportant un résumé exécutif et le détail technique des résultats.	MVP
RF-28	Associer à chaque résultat une recommandation de correction reliée, lorsque possible, à une source vérifiable.	MVP
RF-29	Exporter le rapport en PDF et/ou HTML.	MVP
RF-30	Comparer les résultats d'un scan avec ceux d'un scan précédent sur la même cible.	MVP simplifié si le temps le permet, sinon Post-MVP

6.5 Module Application et connaissances

Code	Besoin	Statut
RF-31	Fournir une interface en ligne de commande (CLI) permettant l'automatisation des opérations principales (import, scan, rapport).	MVP
RF-32	Fournir une application desktop Windows permettant l'utilisation interactive complète de Tscan.	MVP
RF-33	Maintenir localement une base de connaissances des vulnérabilités (CVE/NVD, CWE, CISA KEV).	MVP (sous-ensemble ciblé)
RF-34	Mettre à jour la base de connaissances lorsqu'une connexion Internet est disponible.	MVP
RF-35	Permettre l'utilisation des fonctions d'import, de corrélation et de reporting sans connexion Internet.	MVP

Chapter 7

Besoins non fonctionnels

7.1 Performance

- **RNF-01** — Un import de résultats externes de taille usuelle (quelques centaines de résultats) doit être traité en quelques secondes.
- **RNF-02** — Un scan actif Tscan sur une cible web unique doit rester borné dans le temps par les paramètres de périmètre définis par l'utilisateur (durée maximale configurable), afin de garantir un usage prévisible.
- **RNF-03** — L'application desktop doit rester réactive (pas de gel d'interface) pendant l'exécution d'un scan ou d'un import, grâce à un traitement asynchrone des opérations longues.

7.2 Sécurité

- **RNF-04** — Toute donnée sensible persistée localement (résultats, preuves, éventuels identifiants de cibles authentifiées en post-MVP) doit être protégée de manière appropriée sur le poste de l'utilisateur.
- **RNF-05** — Toute action de scan actif doit être journalisée (cible, horodatage, périmètre, utilisateur) à des fins de traçabilité et d'audit.
- **RNF-06** — Le logiciel doit lui-même respecter des pratiques de développement sécurisées (gestion des erreurs, validation des entrées, absence de exécution de code non maîtrisée).

7.3 Portabilité et compatibilité

- **RNF-07** — L'application desktop et la CLI doivent fonctionner sur les versions de Windows couramment utilisées en environnement professionnel au moment du projet.
- **RNF-08** — Le packaging de l'application doit permettre une installation simple, sans dépendance manuelle complexe pour l'utilisateur final.

7.4 Maintenabilité et évolutivité

- **RNF-09** — L'ajout d'un nouveau format d'import ne doit nécessiter la modification d'aucun composant du moteur de corrélation ou de scoring.

- **RNF-10** — L'ajout d'une nouvelle règle de détection ne doit pas nécessiter de modification du code du moteur central.
- **RNF-11** — Le code source doit respecter des conventions homogènes, être documenté, et couvert par des tests automatisés sur les composants critiques (parsing, corrélation, scoring).

7.5 Utilisabilité

- **RNF-12** — L'interface desktop doit être exploitable par un analyste en cybersécurité sans formation préalable prolongée, tout en restant suffisamment technique pour ne pas masquer les preuves et détails nécessaires à son travail.
- **RNF-13** — Le rapport généré doit être compréhensible par un lecteur non technique dans sa partie résumé exécutif, sans sacrifier la précision technique de la partie détaillée.

7.6 Disponibilité hors-ligne

- **RNF-14** — Les fonctions d'import, de corrélation, de validation locale, de scoring et de génération de rapport doivent être opérationnelles sans connexion Internet, dès lors que la base de connaissances locale a été téléchargée au moins une fois.
- **RNF-15** — L'absence de connexion Internet doit être gérée explicitement (message clair à l'utilisateur) plutôt que de provoquer une erreur silencieuse ou un blocage.

Chapter 8

Contraintes

8.1 Contraintes de temps et de ressources

Je développe ce projet seul, sur une durée d'environ deux mois et demi, à raison d'environ huit heures de travail par jour. Mon niveau de compétence général dans les domaines techniques concernés est intermédiaire. Ces contraintes ont directement déterminé le resserrement du périmètre du MVP décrit au chapitre 2, en particulier la limitation du moteur de scan actif à cinq familles de vulnérabilités et au seul périmètre web.

8.2 Contraintes techniques

- **Plateforme cible** — l'application principale doit fonctionner en environnement Windows, ce qui contraint les choix technologiques ultérieurs (chapitre Architecture et technologies).
- **Fonctionnement hybride** — une partie des fonctionnalités doit rester utilisable hors connexion Internet, ce qui impose une architecture séparant clairement les traitements locaux des traitements dépendants de sources externes.
- **Aucune base existante** — l'ensemble de l'architecture, de la base de données et du code doit être conçu depuis zéro, sans contrainte de compatibilité avec un système préexistant.

8.3 Contraintes organisationnelles

- Le projet doit avancer par étapes validées successivement, sans anticiper de décision architecturale ou technologique majeure avant la fin des phases d'étude et de définition du produit.
- Le projet doit rester documenté progressivement (README, décisions techniques, documentation utilisateur et développeur) plutôt qu'en fin de stage.
- Le projet doit être versionné sur un dépôt Git, avec un historique de commits explicite et cohérent.

8.4 Contraintes légales et éthiques

- Toute fonctionnalité de scan actif ne doit être utilisée que sur des cibles pour lesquelles une autorisation explicite existe. Cette contrainte est structurante et non négociable pour l'ensemble du projet (voir chapitre 9).

- L'intégration de sources de connaissances externes (CVE/NVD, CWE, CISA KEV) doit respecter les conditions d'accès et de réutilisation propres à chacune de ces sources.

Chapter 9

Exigences de sécurité

Tscan est un outil de sécurité offensive. À ce titre, les exigences de sécurité qui encadrent son fonctionnement sont considérées comme des exigences de premier rang, au même niveau que les besoins fonctionnels, et non comme une simple contrainte annexe.

9.1 Autorisation et périmètre

Principe fondamental
Aucune fonctionnalité de scan actif de Tscan ne doit pouvoir s'exécuter sur une cible sans que l'utilisateur ait explicitement confirmé disposer de l'autorisation nécessaire pour cette cible.

- **ES-01** — L'utilisateur doit explicitement définir et confirmer la cible autorisée avant tout scan actif.
- **ES-02** — Le périmètre d'un scan doit pouvoir être limité en domaine, en profondeur, en durée et en types de tests autorisés.
- **ES-03** — Un mode sécurisé, privilégiant les tests non destructifs, doit être activé par défaut.
- **ES-04** — Les familles de vulnérabilités dont la validation active présenterait un risque significatif pour la disponibilité ou l'intégrité de la cible (par exemple SSRF, désérialisation) sont explicitement exclues du MVP, comme indiqué au chapitre 2.

9.2 Traçabilité et journalisation

- **ES-05** — Chaque scan actif doit être journalisé : cible, horodatage de début et de fin, périmètre appliqué, résultats produits.
- **ES-06** — Chaque correction manuelle d'un statut de vulnérabilité doit être tracée (valeur automatique initiale, valeur corrigée, auteur, date).
- **ES-07** — Le résultat brut d'origine de tout import externe doit être conservé sans modification, indépendamment du résultat normalisé et enrichi produit par Tscan.

9.3 Protection des données collectées

- **ES-08** — Les preuves collectées (requêtes, réponses, captures) peuvent contenir des informations sensibles relatives à la cible ; leur stockage local doit être traité avec la même rigueur que toute donnée sensible.

- **ES-09** — Aucune preuve ni aucun résultat de scan ne doit être transmis à un tiers ou à un service externe sans action explicite de l'utilisateur.

9.4 Sécurité de l'outil lui-même

- **ES-10** — Le code de Tscan doit valider et neutraliser correctement les entrées utilisées pour construire les requêtes de test envoyées à une cible, afin d'éviter que Tscan lui-même n'introduise une faille exploitable.
- **ES-11** — Les mécanismes de mise à jour de la base de connaissances et des règles doivent vérifier l'intégrité des données récupérées avant intégration.
- **ES-12** — Toute dépendance logicielle tierce utilisée par Tscan doit être choisie et suivie avec la même vigilance que celle appliquée aux cibles scannées par Tscan lui-même.

Chapter 10

Architecture prévisionnelle

10.1 Vue d'ensemble

L'architecture de Tscan est organisée en trois couches strictement séparées, conçues pour respecter le principe énoncé au chapitre 1 : **Moteur** $\neq$ **Base de connaissances** $\neq$ **Règles**.

- (1) **Présentation** — deux clients indépendants : une interface en ligne de commande (CLI) pour l'automatisation, et une application desktop pour l'usage interactif.
- (2) **Cœur Tscan** — une bibliothèque partagée regroupant l'orchestrateur de scan, le module d'import et de corrélation, et le module de reconnaissance/détection/validation piloté par le système de règles.
- (3) **Stockage local** — base de données (résultats, preuves, historique), fichiers de règles versionnés, et cache local de la base de connaissances.

Un composant transverse, le *gestionnaire de mise à jour*, est seul autorisé à effectuer des appels réseau vers les sources externes de connaissances (CVE/NVD, CWE, CISA KEV). Le reste du cœur fonctionne exclusivement sur les données locales, ce qui garantit structurellement le respect de l'exigence de fonctionnement hors-ligne (RNF-14, RNF-15), plutôt que de la traiter comme une vérification a posteriori.

10.2 Décision d'architecture — Partage de la logique entre CLI et application desktop

La CLI et l'application desktop ne doivent pas dupliquer la logique d'import, de corrélation, de validation et de scan : une telle duplication doublerait le risque de bug, le temps de développement et le risque d'incohérence entre les deux interfaces, ce qui est incompatible avec les contraintes du chapitre 8.

Retenu : un cœur Tscan sous forme de bibliothèque partagée, consommée directement par la CLI et par l'application desktop, sans communication réseau entre elles. Ce choix élimine toute duplication de logique métier avec une complexité minimale, tout en n'empêchant pas une évolution ultérieure vers une exposition en service si Tscan devait devenir multi-utilisateurs (voir chapitre 10, section « Évolution future »).

Ce choix contraint fortement la sélection technologique (chapitre suivant) : il faut un écosystème permettant de construire une bibliothèque consommable à la fois par une CLI et par une application desktop native Windows, sans pont technique lourd entre les deux.

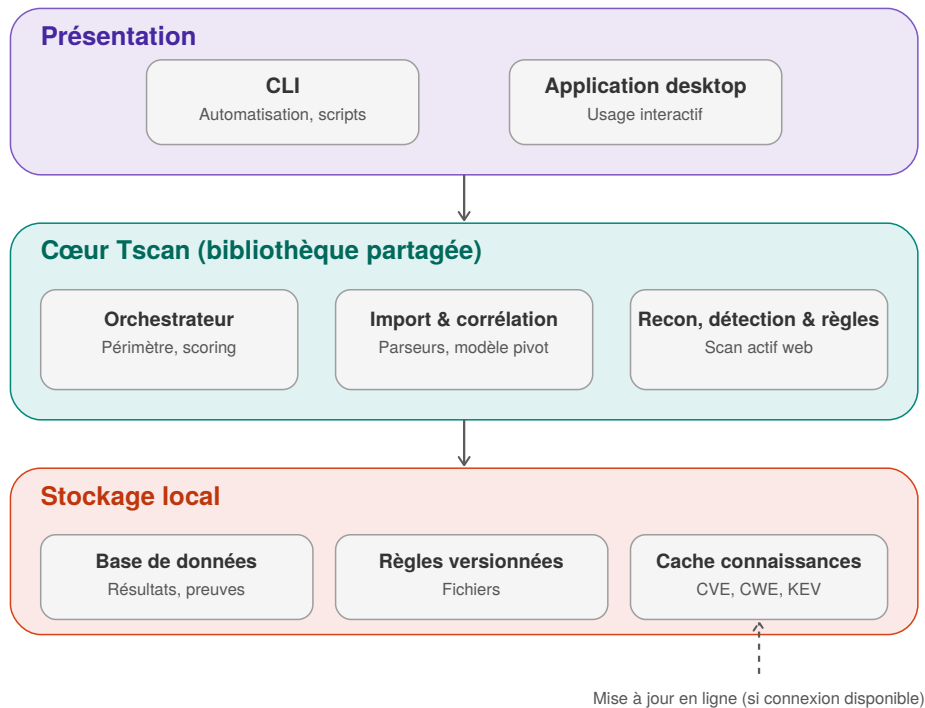


Figure 10.1: Architecture globale de Tscan : présentation, cœur partagé et stockage local.

10.3 Décision d'architecture — Découplage moteur / règles / connaissances

Les consignes initiales du projet exigent explicitement que l'ajout d'une règle de détection ou d'une nouvelle source de connaissance ne nécessite pas de modifier le code du moteur central.

Retenu : des fichiers de règles versionnés pour le MVP, structurés selon les champs définis au chapitre 1 (identifiant, catégorie, gravité, conditions de détection, méthode de validation, preuves attendues, recommandations, références, version), chargés dynamiquement par le moteur au démarrage. Cette approche atteint l'objectif de découplage à un coût de développement minimal et s'intègre naturellement à un dépôt Git ; le passage à une gestion en base de données avec interface d'administration reste une évolution naturelle post-MVP.

La même logique de découplage est appliquée à la base de connaissances : un cache local structuré, alimenté exclusivement par le gestionnaire de mise à jour, sans dépendance directe entre le code du moteur et le contenu des données de connaissance.

10.4 Modèle de données pivot

Tout résultat entrant dans Tscan — qu'il provienne d'un import externe (Nuclei, ZAP, Nessus) ou du moteur de scan actif de Tscan — est converti vers un modèle de données unique, désigné *Finding*, avant tout traitement de corrélation, de validation ou de scoring. Ce modèle pivot garantit que le moteur de corrélation ne dépend jamais du format d'origine d'un résultat, ce qui permet l'ajout d'un nouveau format d'import (RNF-09) sans modification du moteur.

Chaque *Finding* conserve une référence vers le résultat brut d'origine (ES-07), et est associé à un historique de statuts (automatique puis, le cas échéant, corrigé manuellement — RF-12).

10.5 Gestion des preuves

Les preuves associées à un résultat (requêtes, réponses, captures) sont stockées sous forme de fichiers référencés par leurs métadonnées en base de données, plutôt que directement en base. Cette approche évite de saturer la base de données, facilite l'export ou l'archivage d'un dossier de scan complet, et simplifie la mise en œuvre de la protection des données collectées (ES-08, ES-09).

Chapter 11

Technologies envisagées

11.1 Langage du cœur

Retenu : Python.

Python est le langage que je maîtrise déjà, ce qui élimine tout risque d'apprentissage cumulé avec le développement d'une interface graphique. Il constitue par ailleurs l'un des écosystèmes les plus riches pour la sécurité offensive (bibliothèques HTTP, analyse HTML, automatisation de sondes), ce qui réduit directement le temps de développement du moteur de détection — la partie la plus critique du planning (chapitre 6, bloc 3). Ce choix s'applique de façon homogène au cœur, à la CLI et à l'application desktop, afin de conserver un unique langage sur l'ensemble du projet et de simplifier l'architecture à cœur partagé définie au chapitre 10.

11.2 Application desktop

Retenu : PySide6 (Qt pour Python).

PySide6 est un framework d'interface graphique natif et mature, dont les composants d'affichage de données tabulaires (listes filtrables, arbres de résultats) sont directement adaptés à l'usage principal de l'application : la consultation d'un grand nombre de résultats de scan avec leurs preuves associées. Il permet de rester dans un seul langage plutôt que d'ajouter une couche web supplémentaire à apprendre en parallèle, ce qui est prioritaire pour un projet solo sans expérience préalable de développement d'interface graphique.

Point de vigilance retenu pour le planning
--

L'apprentissage de PySide6 (signaux/slots, agencement des vues) constitue le principal risque technique lié à ce choix, dans la mesure où je n'ai aucune expérience préalable en développement d'interface graphique. Ce risque est explicitement intégré au planning du chapitre suivant plutôt que traité comme un acquis.
--

11.3 Synthèse de la pile technologique retenue

Composant	Choix retenu	Justification
Langage du cœur	Python	Seul langage maîtrisé ; permet un cœur partagé sans pont technique (chapitre 10).
Interface desktop	PySide6 (Qt)	Composants natifs adaptés à l’affichage de données tabulaires denses ; un seul langage à apprendre.
Interface en ligne de commande	Typer	Basé sur les annotations de type Python, aide générée automatiquement, syntaxe alignée sur la forme cible <code>tscan scan <cible></code> .
Base de données	SQLite (via SQLAlchemy)	Application mono-utilisateur locale : aucun serveur à administrer, fichier unique facilement sauvegardé ; migration vers un moteur serveur envisageable en cas d’évolution multi-utilisateurs post-MVP.
Requêtes HTTP (reconnaissance, scan)	httpx	API proche des bibliothèques HTTP synchrones usuelles, support HTTP/2, et mode asynchrone disponible sans réécriture si une parallélisation des sondes devient nécessaire.
Analyse HTML	BeautifulSoup4 + lxml	Bibliothèques standards, robustes et documentées pour l’analyse de pages web durant la reconnaissance.
Format des règles de détection	YAML	Lisible, éditable manuellement, cohérent avec les conventions observées lors de l’étude de l’existant (chapitre 4).
Empaquetage Windows	PyInstaller + Inno Setup	Production d’un exécutable Windows autonome et d’un installeur, sans dépendance manuelle pour l’utilisateur final.
Tests automatisés	pytest	Standard de facto de l’écosystème Python, intégration simple à un pipeline de vérification continue.

11.4 Autres points de vigilance

- Le mode asynchrone de httpx ne sera introduit que si le besoin de paralléliser des sondes se confirme pendant le développement du moteur de scan ; un usage synchrone simple reste la première approche, pour ne pas ajouter de complexité non justifiée.
- L’empaquetage via PyInstaller doit être testé dès que l’application desktop atteint un état minimal fonctionnel, afin de détecter tôt d’éventuels problèmes de dépendances manquantes à l’exécution, plutôt qu’en toute fin de projet.

Chapter 12

Méthodologie de développement

12.1 Principe général

Je conduis ce projet par étapes successives, chacune validée avant de passer à la suivante plutôt qu'en avançant simultanément sur plusieurs fronts. Une étape n'est considérée comme terminée que lorsque son objectif est atteint, que les tests nécessaires ont été effectués, et que la documentation associée est à jour. Ce principe, déjà appliqué à la construction de ce cahier des charges lui-même (étude de l'existant, définition du produit, architecture, technologies, chacune validée avant la suivante), se poursuit à l'identique pendant la phase de développement.

12.2 Développement incrémental

Chaque fonctionnalité est développée, testée et validée avant de passer à la fonctionnalité suivante, plutôt que de construire plusieurs modules en parallèle de façon inachevée. L'ordre de développement privilégie les blocs qui réduisent le plus tôt possible l'incertitude technique la plus importante — en particulier le moteur de scan actif (chapitre 6, bloc 3), identifié comme le point le plus risqué du planning.

12.3 Gestion des blocages

En cas d'échec ou de blocage sur une étape, je reste sur cette étape jusqu'à sa résolution plutôt que de contourner silencieusement le problème ou de construire au-dessus d'une base non fiable. La résolution suit systématiquement les mêmes étapes : identification du problème, analyse de sa cause, recensement des solutions possibles, choix, correction, test, vérification.

12.4 Documentation continue

La documentation (README, décisions techniques, documentation utilisateur et développeur, changelog) est tenue à jour progressivement au fil du développement, et non rédigée en fin de projet. Ce cahier des charges lui-même reste un document vivant, mis à jour à chaque décision structurante.

12.5 Gestion de version

Le projet est versionné sur un dépôt Git dès la phase de mise en place (chapitre 13), avec des messages de commit explicites et une structure de dépôt cohérente avec l'architecture définie au chapitre 10.

12.6 Revues régulières

Une revue d'architecture (cohérence, modularité, dette technique) et une revue de code (qualité, lisibilité, tests, sécurité) sont effectuées à l'issue de chaque bloc fonctionnel du planning, plutôt qu'une seule fois en fin de projet. Toute dérive constatée par rapport à l'architecture du chapitre 10 est traitée avant de poursuivre.

Chapter 13

Planning

13.1 Cadrage général

Le projet se déroule sur environ deux mois et demi, à raison d'environ huit heures de travail par jour, cinq jours par semaine, soit environ onze semaines de travail effectif. Le planning ci-dessous découpe ces onze semaines en blocs correspondant aux phases déjà présentées dans ce document. Les deux premières semaines, consacrées à l'étude de l'existant, à la définition du produit, au présent cahier des charges et aux choix d'architecture et de technologies, sont déjà réalisées au moment de la rédaction de ce chapitre.

13.2 Répartition par semaine

Période	Objectif	Livrables et critères de validation
S1	Étude de l'existant, définition du produit	Synthèse comparative, positionnement de Tscan, MVP fonctionnel défini. <i>Terminé.</i>
S2	Cahier des charges, architecture, technologies	Présent document (chapitres 1 à 11) validé. <i>Terminé.</i>
S3	Mise en place du projet	Dépôt Git structuré, environnement Python configuré, squelette de la bibliothèque cœur et de la base de données, premiers tests automatisés, premières bases de PySide6 explorées en parallèle pour lisser la courbe d'apprentissage.
S4	Bloc import & modèle pivot	Parseurs Nuclei et ZAP fonctionnels, modèle <i>Finding</i> , conservation du résultat brut, statuts de validation. Tests unitaires sur les parseurs.
S5–S6	Bloc corrélation, validation et scoring	Pré-filtrage contextuel, corrélation multi-sources, scoring de confiance explicable, moteur de règles chargeant des fichiers versionnés. Scénario A démontrable en CLI.
S7	Bloc reconnaissance, détection (familles simples) et base de connaissances	Base de connaissances CVE/NVD (interrogée à la demande, avec cache local), CISA KEV (synchronisation complète), table de référence CWE minimale, gestionnaire de mise à jour isolé (RF-33, RF-34, ajouté à ce bloc suite à une revue d'avancement — absent du planning initial). Reconnaissance web, Security Misconfiguration, composants vulnérables connus (RF-18, désormais adossé à la base de connaissances réelle), Broken Access Control basique.
S8	Bloc détection (familles restantes) et validation active	XSS réfléchi, CSRF, validation active non destructive pour les cinq familles, contrôle de périmètre. Injection SQL error-based traitée en bonus si l'avancement le permet.
<i>Jalon de sécurité — fin S8 : scénarios A et B démontrables intégralement en CLI.</i>		
S9	Bloc reporting et finalisation CLI	Génération de rapport (résumé exécutif + détails techniques), recommandations sourcées, CLI complète, tests d'intégration du cœur.
S10	Application desktop	Interface PySide6 consommant le cœur déjà validé (import, résultats, preuves, lancement de scan, rapport).
S11	Intégration finale et démonstration	Durcissement, tests de bout en bout, documentation finalisée, préparation et répétition de la démonstration (scénarios A et B).

13.3 Jalon de sécurité

La fin de la semaine 8 constitue un jalon explicite : à cette date, les scénarios de démonstration A et B (chapitre 14) doivent être fonctionnels au moins en ligne de commande, indépendamment de l'état d'avancement de l'application desktop. Ce jalon garantit qu'un retard sur l'apprentissage ou l'intégration de PySide6 (risque identifié au chapitre 11) ne compromette pas la capacité à présenter le cœur fonctionnel de Tscan.

13.4 Suivi d'avancement

L'avancement réel est comparé à ce planning à l'issue de chaque semaine. Tout retard constaté est signalé explicitement plutôt que dissimulé, avec une proposition de réorganisation du planning restant, en conservant la priorité énoncée au chapitre 2 : la qualité du MVP prime sur le nombre de fonctionnalités livrées.

Écart constaté et corrigé — revue de fin de semaine 6

La revue de code de fin de bloc corrélation/validation/scoring a révélé que les besoins RF-33 et RF-34 (base de connaissances CVE/NVD, CISA KEV) étaient marqués MVP dans ce cahier des charges sans jamais avoir reçu de semaine dédiée dans ce planning. La détection de composants vulnérables (RF-18, semaine 7) en dépend directement. Plutôt que d'implémenter un espace réservé, ce bloc est intégré à la semaine 7 (ci-dessus), sans modifier la durée totale du planning, au prix d'une semaine 7 plus chargée que prévu initialement.

Chapter 14

Critères de réussite du MVP

Je considère le MVP réussi si, à l'issue des onze semaines de planning, Tscan permet de démontrer :

- **Scénario A** — l'import d'un résultat Nuclei et d'un résultat ZAP portant sur une même cible, leur corrélation, l'attribution de statuts avec preuves à l'appui, et la génération d'un rapport.
- **Scénario B** — le lancement d'un scan direct sur une cible web autorisée, incluant reconnaissance, détection sur les cinq familles retenues, validation active, scoring, et génération d'un rapport.
- Une architecture conforme au chapitre 10, permettant l'ajout d'un sixième format d'import ou d'une sixième famille de détection sans modification du moteur central.
- Un dépôt Git structuré, une documentation technique et utilisateur à jour, et une suite de tests automatisés couvrant les composants critiques (parseurs, corrélation, scoring).

Ces critères reprennent et formalisent ceux déjà énoncés lors de la définition du produit (chapitre 2).

Chapter 15

Limites explicites du MVP

Afin d'éviter toute ambiguïté sur ce que Tscan couvre à l'issue du stage, je liste ici explicitement ce qui est volontairement exclu du MVP :

- Le moteur de scan actif ne couvre que le périmètre web (HTTP/HTTPS) et cinq familles de vulnérabilités (Security Misconfiguration, composants vulnérables connus, XSS réfléchi, Broken Access Control basique, CSRF), l'injection SQL error-based restant un objectif bonus non garanti.
- Le scan réseau (hors web), le scan authentifié, le scan d'API dédié, ainsi que les familles à validation complexe ou risquée (SSRF, désérialisation) ne sont pas traités.
- L'apprentissage automatique à partir des corrections des analystes n'est pas implémenté ; le scoring de confiance du MVP repose exclusivement sur des règles déterministes.
- Tscan est conçu pour un usage mono-utilisateur, local à un poste Windows ; aucun mécanisme de collaboration multi-utilisateurs ou de déploiement réseau n'est prévu dans le MVP.
- La comparaison entre scans successifs sur une même cible n'est traitée que sous une forme simplifiée, si le temps disponible le permet.
- Le parseur Nessus est traité en priorité secondaire ; son absence éventuelle du MVP n'empêche pas la démonstration des scénarios A et B, qui reposent sur Nuclei et ZAP.

Chapter 16

Perspectives post-stage

Les évolutions suivantes ne sont pas développées pendant le stage, mais l'architecture définie au chapitre 10 — en particulier le découplage moteur/règles/connaissances et le modèle de données pivot — est conçue pour ne pas en rendre l'ajout ultérieur inutilement difficile :

- Apprentissage à partir des corrections effectuées par les analystes, pour faire évoluer automatiquement le scoring de confiance et les règles.
- Système d'alertes et surveillance continue d'une cible entre deux scans planifiés.
- Élargissement du scan actif au périmètre réseau, à l'authentification, et aux spécifications d'API.
- Ajout de nouvelles sources de renseignement (par exemple GitHub Security Advisories) et de nouveaux formats d'import (par exemple SARIF en entrée, en complément de l'export déjà envisagé).
- Évolution de la gestion des règles vers une interface d'administration dédiée, en remplacement des fichiers versionnés du MVP.
- Collaboration multi-utilisateurs et déploiement en réseau, si l'usage à l'ANTIC ou une éventuelle commercialisation le justifient.